

PREMIUM TRAINING GUIDE

VictoriaMetrics

as an LLM Observability Backend

Concrete setup, MetricsQL, dashboards, alerts and sovereign architecture

Samuel Desseaux

Principal Observability Engineer & Trainer

Practical guide for production LLM workloads

Table of Contents

Module 1 : Why VictoriaMetrics for LLM workloads	3
Module 2 : Complete architecture VM + OTel	5
<i>Diagram 1 : Full architecture overview</i>	6
Module 3 : Single Node vs Cluster - choosing your deployment mode	8
<i>Diagram 2 : Single vs Cluster comparison</i>	9
Module 4 : Step-by-step installation and configuration	11
Module 5 : LLM metrics taxonomy	16
<i>Diagram 3 : Complete LLM metrics taxonomy</i>	17
Module 6 : Python instrumentation - pushing metrics	20
Module 7 : MetricsQL - essential queries for LLM observability	25
<i>Diagram 4 : MetricsQL vs PromQL - key features</i>	26
Module 8 : Grafana dashboards - structure and panels	31
<i>Diagram 5 : Recommended Grafana LLM dashboard layout</i>	32
Module 9 : VMAAlert - production alert rules	36
Module 10 : Recording rules - pre-aggregating cost metrics	40
Module 11 : Optimization, retention and tuning	43
Module 12 : Security, air-gap and compliance	46
Module 13 : Practical labs	48
Module 14 : References and resources	51
Module 15: Production case studies	31
Module 16: Migrating from Prometheus	33
Module 17: SLO and SLA for LLM workloads	35
Module 18: FAQ and troubleshooting	37
Appendix A: Glossary	39
Appendix B: MetricsQL cheat sheet	41
Appendix C: Production go-live checklist	43

1. Why VictoriaMetrics for LLM Workloads

1.1 The specific challenge of LLM metrics

LLM workloads in production generate an atypical metrics profile that pushes classic monitoring backends to their limits:

LLM Characteristic	Impact on the metrics backend
High cardinality	model * provider * tenant * pipeline = thousands of timeseries. Prometheus saturates quickly.
Token histograms	Wide distributions (10 to 100k tokens/request). Very spread-out buckets.
USD cost metrics	Financial metrics requiring float precision. Aggregations by accounting period.
Long retention	Cost audits, contractual SLAs: 1-3 year retention is often required.
Ingestion bursts	A completion batch can generate 10x the normal write rate in seconds.
RAG quality metrics	Floats, distributions, scores: write-intensive, frequently read for alerting.

1.2 VictoriaMetrics vs Prometheus - head-to-head

Criteria	Prometheus	VictoriaMetrics
Cardinality	Degrades above 500k active series	Stable up to 10M+ active series
Compression	~4:1 ratio	~10:1 ratio (20:1 with long retention)
Write throughput	~200k samples/s per instance	>1M samples/s on a standard instance
Long retention	Degrades after 90 days	Performant over 2+ years without manual compaction
Query language	PromQL	MetricsQL (superset: anomaly_score, median_over_time...)
Native OTLP	Via Collector only	Direct OTLP endpoint in Single mode (port 4318)
Deployment	Multiple recommended components	Single binary in Single mode
RAM footprint	4 GB RAM / 500k series	1 GB RAM / 500k series (4x less)

Criteria	Prometheus	VictoriaMetrics
Migration	N/A	100% Prometheus-compatible API: zero dashboard migration

1.3 Learning objectives

By the end of this guide, you will be able to:

- Deploy VictoriaMetrics in Single mode in 15 minutes for an LLM development environment
- Choose between Single and Cluster mode based on your actual LLM workload
- Configure the OTel Collector -> VictoriaMetrics pipeline for LLM metrics collection
- Instrument a Python/FastAPI application to emit the right LLM metrics
- Write MetricsQL queries for 8 essential use cases (cost, quality, latency, GPU)
- Build a Grafana LLM dashboard with 4 functional rows and real-time alerts
- Configure VMAalert with production-grade alert rules for LLM degradations
- Optimize retention, storage and recording rules to minimize infra costs

2. Complete Architecture VM + OTel for LLM

2.1 Overview

The architecture below covers the full metrics collection path from LLM sources to dashboards and alerts. It is deployable in full air-gap mode.

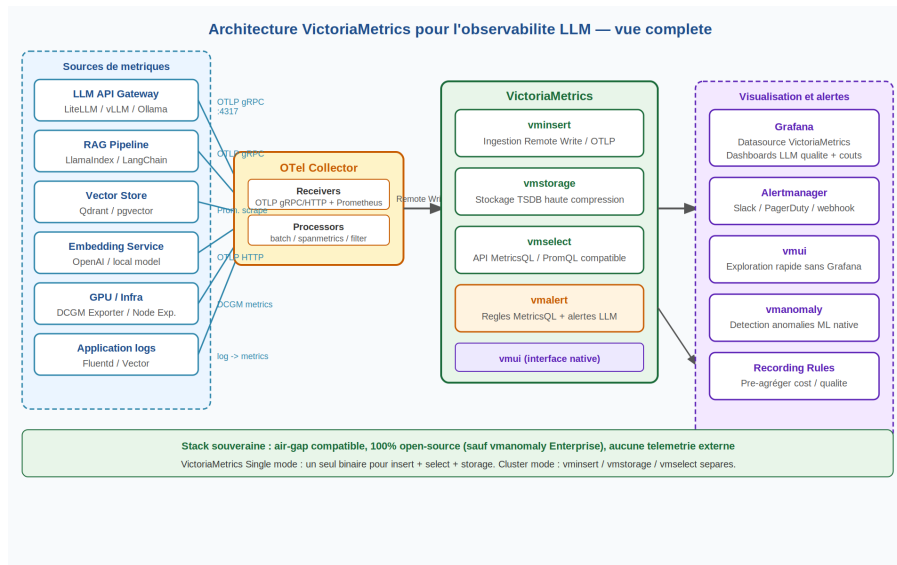


Diagram 1 : Complete VictoriaMetrics architecture for LLM observability - air-gap compatible

Key architectural points

All sources emit to the OTel Collector via OTLP (gRPC port 4317 or HTTP port 4318)
 The Collector filters, enriches (k8s labels), derives metrics from traces (spanmetrics), and exports
 VictoriaMetrics receives via Remote Write. In Single mode, it also accepts OTLP directly.
 VMAAlert evaluates MetricsQL rules and sends alerts to Alertmanager
 Grafana uses VictoriaMetrics as a datasource (Prometheus-compatible, zero reconfiguration)
 vmanomaly (Enterprise) adds ML-based anomaly detection on LLM metrics

2.2 Components and roles

Component	Port(s)	Role for LLM observability
OTel Collector	4317 (gRPC), 4318 (HTTP)	Normalises and routes all metrics. spanmetrics derives metrics from RAG traces.
VictoriaMetrics Single	8428	TSDB storage, MetricsQL API, Remote Write, direct OTLP, built-in vmui

Component	Port(s)	Role for LLM observability
vmagent	8429	Distributed Prometheus scraping. Ideal for DCGM Exporter, vLLM /metrics, Ollama
vmalert	8880	Evaluation of MetricsQL alert rules. Talks to Alertmanager.
Alertmanager	9093	Alert routing: Slack, PagerDuty, email, webhook. Deduplication and silencing.
Grafana	3000	Dashboards. Uses the VictoriaMetrics plugin or standard Prometheus datasource.
vmanomaly (Enterprise)	9090	ML anomaly detection (ARIMA, Prophet) on LLM metrics. Optional.

3. Single Node vs Cluster - Choosing Your Mode

3.1 Comparison diagram

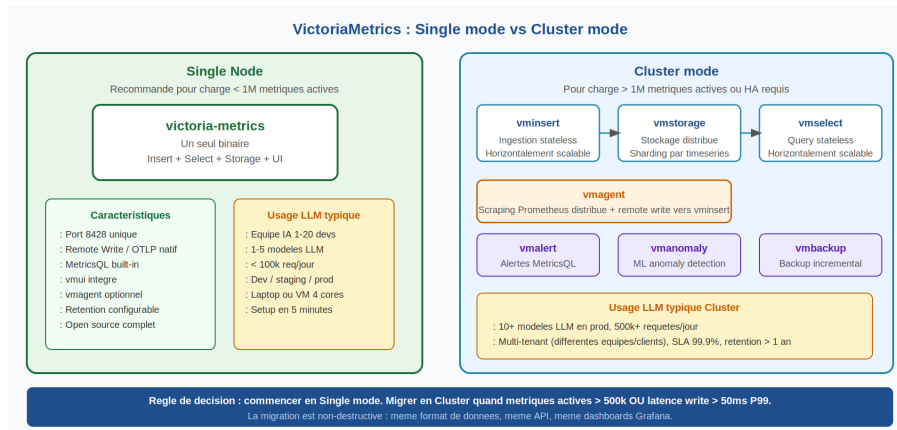


Diagram 2 : Single mode (one binary) vs Cluster mode (3 separate components) - LLM decision criteria

3.2 Decision criteria for an LLM context

Criteria	Single mode	Cluster mode
Active metrics	< 1M (recommended < 500k)	> 500k, theoretically unlimited
LLM requests/s	< 500 rps	> 500 rps or intensive multi-tenant
High availability	Not native (external solutions)	Native: replicas per component
LLM models in prod	1 to 10 models	> 10 models or > 50 tenants
Deployment	1 container / 1 binary	3 Kubernetes deployments minimum
Retention	Configurable, identical	Configurable per tier (hot/warm/cold)
Ops complexity	Very low	Moderate to high
RAM required	1-4 GB depending on load	4 GB+ per component
License	Open source (Apache 2.0)	Open source (vminsert/vmstorage/vmselect)

Migration rule: Single to Cluster

Migrate when: active metrics > 500k OR P99 write latency > 50ms OR HA SLA required.
Migration is non-destructive: same data format, same API, same Grafana dashboards.

Procedure: vmstorage snapshot -> restore on new cluster. Zero downtime possible with vmagent.

4. Installation and Configuration Step by Step

4.1 Docker Compose - full stack in 5 minutes

Development or lightweight production stack: VictoriaMetrics Single + OTel Collector + Grafana + VMAAlert.

```
# docker-compose.yml
version: '3.8'
services:

  victoriametrics:
    image: victoriametrics/victoria-metrics:v1.99.0
    command:
      - '--storageDataPath=/var/lib/victoria-metrics-data'
      - '--retentionPeriod=12' # 12 months retention
      - '--httpListenAddr=:8428'
      - '--search.maxQueryLen=16384' # Long MetricsQL queries
    ports: ['8428:8428']
    volumes:
      - vm-data:/var/lib/victoria-metrics-data

  otel-collector:
    image: otel/opentelemetry-collector-contrib:0.99.0
    command: ['--config=/etc/otel-collector-config.yaml']
    volumes:
      - ./otel-collector-config.yaml:/etc/otel-collector-config.yaml
    ports:
      - '4317:4317' # OTLP gRPC
      - '4318:4318' # OTLP HTTP
    depends_on: [victoriametrics]

  vmagent:
    image: victoriametrics/vmagent:v1.99.0
    command:
      - '--promscrape.config=/etc/prometheus.yml'
      - '--remoteWrite.url=http://victoriametrics:8428/api/v1/write'
    volumes:
      - ./prometheus.yml:/etc/prometheus.yml

  vmaalert:
    image: victoriametrics/vmaalert:v1.99.0
    command:
      - '--datasource.url=http://victoriametrics:8428'
      - '--remoteWrite.url=http://victoriametrics:8428/api/v1/write'
      - '--notifier.url=http://alertmanager:9093'
      - '--rule=/etc/alerts/*.yaml'
    volumes:
      - ./alerts:/etc/alerts

  alertmanager:
    image: prom/alertmanager:v0.27.0
```

```

volumes: ['./alertmanager.yml:/etc/alertmanager/alertmanager.yml']
ports: ['9093:9093']

grafana:
image: grafana/grafana:10.4.0
environment:
- GF_SECURITY_ADMIN_PASSWORD=admin
- GF_INSTALL_PLUGINS=victoriametrics-metrics-datasource
ports: ['3000:3000']

volumes:
vm-data: {}

```

4.2 OTEL Collector configuration for LLM metrics

```

# otel-collector-config.yaml
receivers:
  otlp:
  protocols:
    grpc: { endpoint: 0.0.0.0:4317 }
    http: { endpoint: 0.0.0.0:4318 }
  prometheus:
  config:
    scrape_configs:
      - job_name: vllm
        scrape_interval: 15s
        static_configs: [{ targets: ['vllm-service:8000'] }]
      - job_name: ollama
        scrape_interval: 15s
        static_configs: [{ targets: ['ollama:11434'] }]

processors:
  batch: { timeout: 5s, send_batch_size: 512 }
  k8sattributes:
  extract:
  metadata: [k8s.pod.name, k8s.namespace.name, k8s.deployment.name]
  spanmetrics:
  metrics_exporter: prometheusremotewrite
  latency_histogram_buckets: [50, 100, 200, 500, 1000, 2000, 5000, 10000]
  dimensions:
    - name: gen_ai.request.model
    - name: gen_ai.system
    - name: rag.index.id
    - name: rag.pipeline.id
    - name: rag.tenant.id

exporters:
  prometheusremotewrite:
  endpoint: http://victoriametrics:8428/api/v1/write
  timeout: 30s
  retry_on_failure: { enabled: true, initial_interval: 5s }

service:
  pipelines:
  metrics:

```

```
receivers: [otlp, prometheus]
processors: [k8sattributes, batch]
exporters: [prometheusremotewrite]
traces/metrics:
receivers: [otlp]
processors: [spanmetrics, batch]
exporters: [prometheusremotewrite]
```

4.3 vmagent - scraping DCGM and LLM endpoints

```
# prometheus.yml (used by vmagent)
global:
  scrape_interval: 15s
  external_labels:
    cluster: 'prod-llm'
    environment: 'production'

scrape_configs:
  # DCGM Exporter - NVIDIA GPU metrics
  - job_name: 'dcmg-exporter'
    scrape_interval: 5s
    kubernetes_sd_configs: [{ role: pod }]
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_label_app]
        regex: dcmg-exporter
        action: keep

  # vLLM inference server
  - job_name: 'vllm'
    static_configs:
      - targets: ['vllm-llama:8000', 'vllm-mistral:8001']

  # Ollama (local models)
  - job_name: 'ollama'
    static_configs: [{ targets: ['ollama:11434'] }]

  # LiteLLM proxy
  - job_name: 'litellm'
    static_configs: [{ targets: ['litellm-proxy:4000'] }]
```

5. LLM Metrics Taxonomy

The taxonomy below covers the four dimensions of LLM observability: response quality, performance and latency, costs and tokens, and GPU/CPU infrastructure.

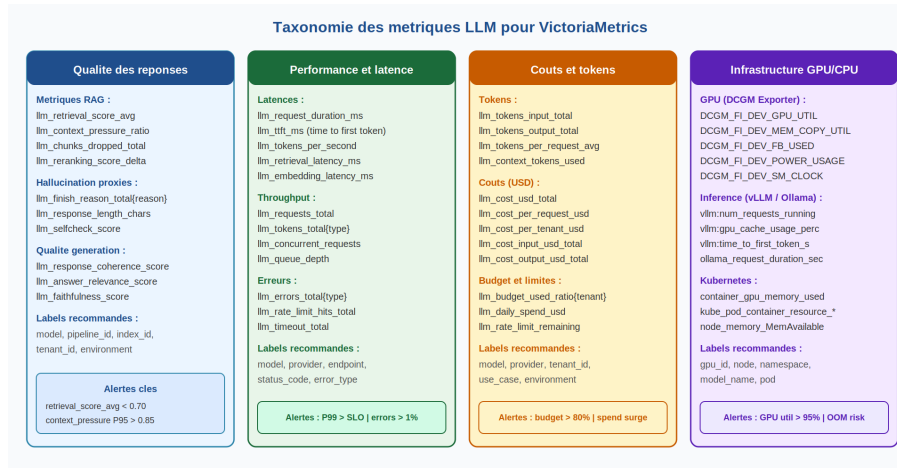


Diagram 3 : Complete LLM metrics taxonomy for VictoriaMetrics - 4 dimensions, recommended labels, alert thresholds

5.1 Recommended naming conventions

- Prefix llm_ for all LLM-specific metrics
- Prefix rag_ for retrieval-specific metrics
- Use DCGM standard names for GPU metrics (DCGM_FI_*)
- Standard suffixes: _total (counter), _seconds/_ms (latency histogram), _ratio (0-1 gauge), _bytes (gauge)

Metric	Prometheus Type	Cardinality	Essential Labels
llm_request_duration_ms	Histogram	Medium	model, provider, pipeline_id, status
llm_tokens_total{type}	Counter	Low	model, provider, type=input output
llm_cost_usd_total	Counter	Low	model, provider, tenant_id, use_case
llm_errors_total	Counter	Medium	model, provider, error_type, status_code
llm_ttf_ms	Histogram	Medium	model, provider, stream=true false
rag_retrieval_score_avg	Gauge	Low	index_id, pipeline_id, strategy
rag_context_pressure_ratio	Histogram	Low	pipeline_id, model, tenant_id
rag_chunks_dropped_total	Counter	Low	pipeline_id, reason
DCGM_FI_DEV_GPU_UTIL	Gauge	Low	gpu, UUID, pod, namespace, node

Metric	Prometheus Type	Cardinality	Essential Labels
vllm:num_requests_running	Gauge	Low	model_name, instance

Cardinality warning: tenant_id label

If you have more than 1000 tenants, the tenant_id label can cause a cardinality explosion. Solution: use a tenant_tier label (free/pro/enterprise) for high-frequency metrics, and keep tenant_id only for cost metrics (llm_cost_usd_total) which have low write frequency.

6. Python Instrumentation - Pushing Metrics

6.1 Setup: two complementary approaches

- Prometheus client library: /metrics endpoint exposed + scraping by vmagent. Simple, battle-tested.
- OTLP via OTel SDK: metrics AND traces in the same pipeline. Recommended for new applications.

```
# Option 1: prometheus_client
pip install prometheus-client

# Option 2: OTel SDK (unified metrics + traces)
pip install opentelemetry-sdk opentelemetry-exporter-otlp-proto-grpc
```

6.2 Prometheus client: core LLM metrics

```
from prometheus_client import Counter, Histogram, Gauge, start_http_server
import time

LLM_REQUESTS = Counter(
    'llm_requests_total', 'Total LLM requests',
    ['model', 'provider', 'pipeline_id', 'status']
)
LLM_DURATION = Histogram(
    'llm_request_duration_ms', 'LLM request duration in ms',
    ['model', 'provider', 'pipeline_id'],
    buckets=[50, 100, 200, 500, 1000, 2000, 5000, 10000, 30000]
)
LLM_TOKENS = Counter(
    'llm_tokens_total', 'Total tokens processed',
    ['model', 'provider', 'type'] # type: input|output
)
LLM_COST = Counter(
    'llm_cost_usd_total', 'Total cost in USD',
    ['model', 'provider', 'tenant_id', 'use_case']
)
LLM_TTFT = Histogram(
    'llm_ttft_ms', 'Time to first token in ms',
    ['model', 'provider'],
    buckets=[50, 100, 200, 500, 1000, 2000, 5000]
)
RAG_RETRIEVAL_SCORE = Gauge(
    'rag_retrieval_score_avg', 'Average retrieval similarity score',
    ['index_id', 'pipeline_id', 'strategy']
)
RAG_CONTEXT_PRESSURE = Histogram(
    'rag_context_pressure_ratio', 'Context window pressure ratio (0-1)',
    ['pipeline_id', 'model'],
    buckets=[0.1, 0.3, 0.5, 0.7, 0.8, 0.85, 0.9, 0.95, 1.0]
)
```

```
# Expose /metrics on port 9090
start_http_server(9090)
```

6.3 Instrumentation decorator - automatic cost tracking

```
import functools

TOKEN_PRICES = {
    'gpt-4o': {'input': 5.0e-6, 'output': 15.0e-6},
    'gpt-4o-mini': {'input': 0.15e-6, 'output': 0.6e-6},
    'claude-3-5-sonnet': {'input': 3.0e-6, 'output': 15.0e-6},
    'mistral-large': {'input': 4.0e-6, 'output': 12.0e-6},
}

def track_llm_call(model, provider, pipeline_id, tenant_id='default',
use_case='chat'):
    def decorator(func):
        @functools.wraps(func)
        async def wrapper(*args, **kwargs):
            labels = {'model': model, 'provider': provider, 'pipeline_id': pipeline_id}
            t0 = time.monotonic()
            status = 'success'
            try:
                result = await func(*args, **kwargs)
                if hasattr(result, 'usage'):
                    in_tok = result.usage.prompt_tokens
                    out_tok = result.usage.completion_tokens
                    LLM_TOKENS.labels(model=model, provider=provider, type='input').inc(in_tok)
                    LLM_TOKENS.labels(model=model, provider=provider, type='output').inc(out_tok)
                    if model in TOKEN_PRICES:
                        p = TOKEN_PRICES[model]
                        cost = in_tok * p['input'] + out_tok * p['output']
                        LLM_COST.labels(model=model, provider=provider,
tenant_id=tenant_id, use_case=use_case).inc(cost)
                    return result
            except Exception as e:
                status = type(e).__name__
                raise
            finally:
                LLM_DURATION.labels(**labels).observe((time.monotonic()-t0)*1000)
                LLM_REQUESTS.labels(**labels, status=status).inc()
            return wrapper
        return decorator

@track_llm_call(model='gpt-4o', provider='openai', pipeline_id='support-rag',
tenant_id='acme')
async def call_llm(messages: list) -> dict:
    return await openai_client.chat.completions.create(model='gpt-4o',
messages=messages)
```

7. MetricsQL - Essential Queries

7.1 MetricsQL features for LLM observability

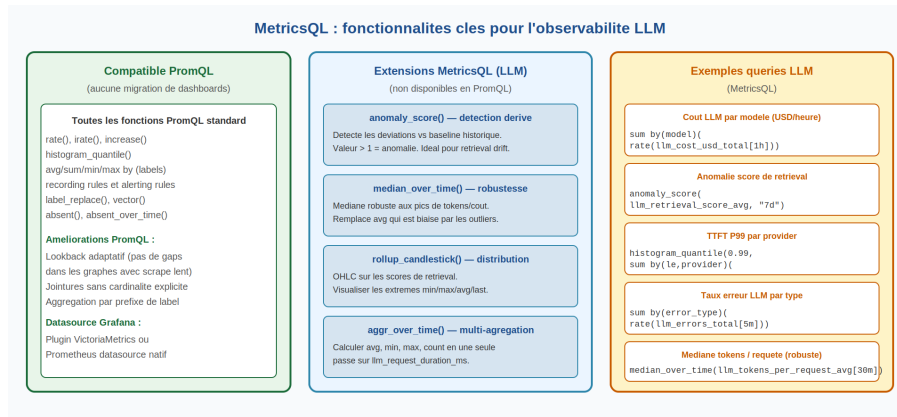


Diagram 4 : MetricsQL vs PromQL - key features for LLM workloads

7.2 Cost and token queries

Hourly LLM cost by model (USD)

```
sum by (model, provider) (increase(llm_cost_usd_total[1h]))
```

Daily cost projection (extrapolated from last 30 minutes)

```
sum by (model) (rate(llm_cost_usd_total[30m])) * 86400
```

Top 10 tenants by cost (last 24h)

```
topk(10, sum by (tenant_id) (increase(llm_cost_usd_total[24h])))
```

Input/output token ratio (detect outlier requests)

```
sum(rate(llm_tokens_total{type='output'}[5m]))
/
sum(rate(llm_tokens_total{type='input'}[5m]))
```

7.3 RAG quality queries

Retrieval score anomaly - index drift detection

```
# anomaly_score: value > 1 = significant deviation vs 7-day baseline
anomaly_score(
  avg by (index_id) (rag_retrieval_score_avg),
  '7d')
```



```
)
```

Context pressure P95 by pipeline

```
histogram_quantile(0.95,  
  sum by (le, pipeline_id) (  
    rate(rag_context_pressure_ratio_bucket[10m])  
  )  
)
```

Requests with dropped chunks (%)

```
sum by (pipeline_id) (rate(rag_chunks_dropped_total[5m]))  
/  
sum by (pipeline_id) (rate(llm_requests_total[5m]))  
* 100
```

7.4 Performance and latency queries

End-to-end P99 latency by model

```
histogram_quantile(0.99,  
  sum by (le, model, provider) (  
    rate(llm_request_duration_ms_bucket[5m])  
  )  
)
```

TTFT P95 - time to first token (critical for streaming)

```
histogram_quantile(0.95,  
  sum by (le, model) (rate(llm_ttft_ms_bucket[5m]))  
)
```

GPU energy efficiency - tokens per watt

```
sum by (model) (rate(llm_tokens_total{type='output'}[5m]))  
/  
avg by (model) (DCGM_FI_DEV_POWER_USAGE)
```

8. Grafana Dashboards - Structure and Panels

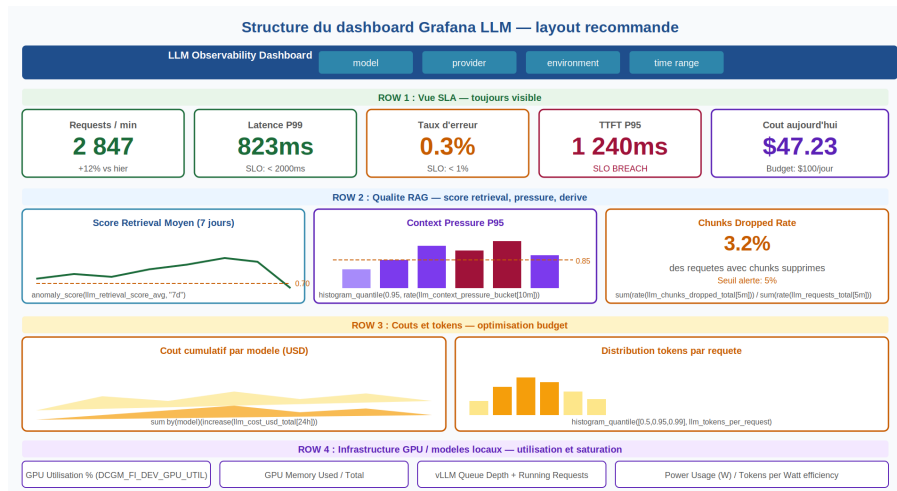


Diagram 5 : Grafana LLM dashboard layout - 4 functional rows with MetricsQL queries

8.1 Datasource configuration

```
# Auto-provision datasource (grafana/provisioning/datasources/vm.yaml)
apiVersion: 1
datasources:
  - name: VictoriaMetrics
    type: victoriametrics-metrics-datasource
    url: http://victoriametrics:8428
    access: proxy
    isDefault: true
    jsonData:
    timeInterval: '15s'
```

8.2 Template variables

```
# Variable 'model': list of active models
label_values(llm_requests_total, model)

# Variable 'provider'
label_values(llm_requests_total{model=~'$model'}, provider)

# Variable 'pipeline_id'
label_values(rag_retrieval_score_avg, pipeline_id)
```

8.3 Recommended panel types per metric

Metric	Panel type	Key options
Requests/min (SLA)	Stat + sparkline	Thresholds: green >0, orange if drift >20%
P99 latency	Stat + threshold	Absolute thresholds per contractual SLO
Error rate	Stat + threshold	Red >1%, orange >0.5%
Retrieval score 7 days	Time series	anomaly_score line + 0.70 threshold
Context pressure P95	Time series + threshold	Red zone above 0.85
Cumulative cost by model	Stacked area time series	Color per model, USD unit
Token distribution	Heatmap or Histogram	Visualise outliers (abnormally long requests)
GPU utilisation	Gauge + threshold	Orange threshold at 80%, red at 95%
vLLM queue depth	Time series	Alert if >10 requests waiting
Anomaly score	Time series	Threshold at 1.0 (significant deviation)

9. VMAAlert - Production Alert Rules

9.1 Complete production ruleset

```
# alerts/llm-quality.yml
groups:
  - name: llm_quality
    interval: 1m
    rules:

    # RAG index drift
    - alert: RAGIndexDrift
      expr: |
        avg by (index_id, pipeline_id) (
          avg_over_time(rag_retrieval_score_avg[1h])
        ) < 0.70
      for: 15m
      labels: { severity: warning, team: ai-platform }
      annotations:
        summary: 'RAG index drift on {{ $labels.index_id }}'
        description: |
          Mean retrieval score = {{ $value | printf "%.2f" }} < threshold 0.70
          for 15m on pipeline {{ $labels.pipeline_id }}.
          Action: check for document updates without re-indexing.
        runbook_url: 'https://wiki.internal/runbooks/rag-index-drift'

    # Context pressure critical
    - alert: RAGContextPressureHigh
      expr: |
        histogram_quantile(0.95,
          sum by (le, pipeline_id) (
            rate(rag_context_pressure_ratio_bucket[10m])
          )
        ) > 0.85
      for: 5m
      labels: { severity: warning, team: ai-platform }
      annotations:
        summary: 'High context pressure P95: {{ $labels.pipeline_id }}'
        description: 'P95 = {{ $value | printf "%.2f" }}. Reduce system prompt or
          increase context window.'

    # High LLM error rate
    - alert: LLMHighErrorRate
      expr: |
        sum by (model, provider) (rate(llm_errors_total[5m]))
        /
        sum by (model, provider) (rate(llm_requests_total[5m]))
        > 0.01
      for: 3m
      labels: { severity: critical, team: ai-platform }
      annotations:
        summary: 'Error rate > 1% on {{ $labels.model }} / {{ $labels.provider }}'
```

```

description: 'Current rate: {{ $value | printf "%.1f" }}%. Check provider
availability.'

# Latency SLO breach
- alert: LLMLatencySLOBreach
expr: |
  histogram_quantile(0.99,
  sum by (le, model, pipeline_id) (
  rate(llm_request_duration_ms_bucket[5m])
  )
  ) > 5000
for: 5m
labels: { severity: critical, team: ai-platform }
annotations:
summary: 'P99 latency SLO breach on {{ $labels.model }}'
description: 'P99 = {{ $value | printf "%.0f" }}ms > SLO 5000ms. Pipeline {{
$labels.pipeline_id }}'

# GPU memory saturation
- alert: GPUMemorySaturation
expr: DCGM_FI_DEV_FB_USED / DCGM_FI_DEV_FB_TOTAL * 100 > 90
for: 2m
labels: { severity: warning, team: mlops }
annotations:
summary: 'GPU {{ $labels.gpu }} memory saturated ({{ $value | printf "%.0f"
}}%)'
description: 'OOM risk. Reduce batch size or model context window.'

# Daily LLM budget
- alert: LLMDailyBudgetWarning
expr: |
  sum by (tenant_id) (
  increase(llm_cost_usd_total[24h])
  ) > 80
labels: { severity: warning, team: billing }
annotations:
summary: 'LLM budget at 80% for tenant {{ $labels.tenant_id }}'
description: '24h cost = ${{{ $value | printf "%.2f" }}}. Threshold: $100/day.'

```

9.2 Alertmanager routing by team

```

# alertmanager.yml
route:
  group_by: ['alertname', 'model', 'pipeline_id']
  group_wait: 30s
  repeat_interval: 12h
  receiver: 'default'
  routes:
    - matchers: [severity='critical']
      receiver: pagerduty
    - matchers: [team='ai-platform']
      receiver: slack-ai-platform
    - matchers: [team='billing']
      receiver: slack-billing

```

```
receivers:
  - name: 'default'
  slack_configs:
    - channel: '#llm-ops-alerts'
      title: '{{ .CommonAnnotations.summary }}'
      text: '{{ .CommonAnnotations.description }}'
    - name: 'pagerduty'
  pagerduty_configs:
    - routing_key: 'YOUR_PAGERDUTY_KEY'
```

10. Recording Rules - Pre-aggregating Cost Metrics

Cost metrics are queried very frequently in dashboards. Recalculating `increase(llm_cost_usd_total[24h])` on every refresh is CPU-intensive. Recording rules pre-compute these aggregations in the background, yielding a 10-100x performance improvement.

```
# alerts/recording-rules.yml
groups:
  - name: llm_recording_rules
    interval: 1m
    rules:

      - record: llm:cost_usd:rate1h
        expr: |
          sum by (model, provider, tenant_id) (
            increase(llm_cost_usd_total[1h])
          )

      - record: llm:cost_usd:rate24h
        expr: |
          sum by (model, provider, tenant_id) (
            increase(llm_cost_usd_total[24h])
          )

      - record: llm:error_rate:ratio5m
        expr: |
          sum by (model, provider) (rate(llm_errors_total[5m]))
          /
          sum by (model, provider) (rate(llm_requests_total[5m]))

      - record: llm:request_duration_ms:p99_5m
        expr: |
          histogram_quantile(0.99,
            sum by (le, model, pipeline_id) (
              rate(llm_request_duration_ms_bucket[5m])
            )
          )

      - record: llm:ttft_ms:p95_5m
        expr: |
          histogram_quantile(0.95,
            sum by (le, model) (rate(llm_ttft_ms_bucket[5m]))
          )

      - record: rag:retrieval_score:avg30m
        expr: |
          avg by (index_id, pipeline_id) (
            avg_over_time(rag_retrieval_score_avg[30m])
          )
```

```
- record: rag:context_pressure:p95_10m
expr: |
  histogram_quantile(0.95,
  sum by (le, pipeline_id) (
  rate(rag_context_pressure_ratio_bucket[10m])
  )
  )

- record: gpu:memory_utilization:ratio
expr: DCGM_FI_DEV_FB_USED / DCGM_FI_DEV_FB_TOTAL
```

Using recording rules in alerts and dashboards

Once defined, use recording rules in alerts and dashboards instead of the full query:

Instead of: `histogram_quantile(0.99, sum by(le,model)(rate(llm_request_duration_ms_bucket[5m])))`

Use: `llm:request_duration_ms:p99_5m`

The query is 50x faster and the result is identical.

11. Optimization, Retention and Tuning

11.1 Retention policy by metric type

Metric type	Recommended retention	Justification	VM flag
USD cost metrics	2-3 years	Accounting audit, contractual SLAs	--retentionPeriod=24m
RAG quality scores	6-12 months	Long-term drift detection	--retentionPeriod=12m
Latency and throughput	3-6 months	Performance trending	--retentionPeriod=6m
GPU metrics	1-3 months	Capacity planning	--retentionPeriod=3m
OTel traces (Jaeger)	7-30 days	Debug, no long-term audit	Jaeger TTL config

11.2 Cardinality optimisation

- Never label by trace_id, request_id or session_id - infinite cardinality.
- Restrict tenant_id to cost metrics only (not for latency or token metrics).
- Group similar models under a model_family label if more than 20 active versions.
- Use adapted histogram buckets for tokens: [10, 100, 500, 1000, 4096, 16384, 65536].

```
# Monitor cardinality in VictoriaMetrics
# Via API: http://victoriametrics:8428/api/v1/status/tsdb

# Top 10 metrics by cardinality (MetricsQL)
topk(10, count by (__name__) ({__name__!=""}))
```

11.3 VictoriaMetrics tuning flags for LLM production

```
victoriametrics \
  --storageDataPath=/var/lib/vm-data \
  --retentionPeriod=12 \
  --httpListenAddr=:8428 \
  --maxInsertRequestSize=32MB \ # Batch LLM writes
  --search.maxQueryLen=16384 \ # Long MetricsQL queries
  --search.maxSamplesPerQuery=1e9 \ # Large time windows for cost audit
  --memory.allowedPercent=60 \ # Leave 40% for the kernel
  --loggerLevel=INFO \
  --loggerFormat=json
```

12. Security, Air-Gap and Compliance

12.1 Authentication and access control

- VictoriaMetrics has no native authentication. Use vmauth (open-source) or an Nginx reverse proxy.
- vmauth supports role-based routing: read-only for Grafana, write-only for OTel Collector, admin for ops.
- Kubernetes NetworkPolicy: restrict access to VM ports to authorised components only.

```
# vmauth config
users:
  - username: grafana
    password: '${GRAFANA_PASSWORD}'
    url_prefix:
      - 'http://victoriametrics:8428/api/v1/query'
      - 'http://victoriametrics:8428/api/v1/query_range'

  - username: otel-collector
    password: '${OTEL_PASSWORD}'
    url_prefix:
      - 'http://victoriametrics:8428/api/v1/write'
```

12.2 Compliance checklist for LLM metrics

Compliance point	Recommended implementation
PII in metrics	Never label with user query content. Use opaque IDs only.
Cost audit (GDPR/NIS2)	3-year retention on llm_cost_usd_total. Monthly CSV export.
Encryption in transit	TLS on all VM endpoints in production (vmauth or reverse proxy).
Multi-tenant isolation	vmauth for per-tenant access segmentation. Grafana Teams + RBAC.
Backup	vmbackup incremental to S3-compatible storage (MinIO for air-gap).
Dashboard access	Grafana with SSO (LDAP, OIDC). Viewer-only for business teams.

13. Practical Labs

Lab 1 - Deploy the stack and ingest first metrics

Goal

Deploy the complete Docker Compose stack (VM + OTel Collector + Grafana + VMAAlert).
Ingest metrics from the provided LLM simulator script. Verify in vmui and Grafana.
Estimated time: 30 minutes | Level: beginner

1. Clone the repo and start the stack

```
git clone https://github.com/your-org/llm-observability-lab
cd llm-observability-lab/labs/victoriametrics-llm
docker compose up -d
docker compose ps # verify all services are UP
```

2. Run the LLM simulator

```
python simulate_llm_metrics.py --rps 10 --models 3 --duration 300
```

3. Check in vmui (<http://localhost:8428/vmui>)

Query: sum by (model) (rate(llm_requests_total[1m])) * 60

4. Import the Grafana dashboard

Import from: llm-observability-lab/labs/victoriametrics-llm/grafana/dashboard-llm-prod.json

Lab 2 - Write 5 MetricsQL queries

Goal

Using metrics generated by the simulator, write the 5 core production queries.
Validate them in vmui, then add them to a Grafana dashboard.
Estimated time: 40 minutes | Level: intermediate

5. Total cost by model over the last 24 hours
6. TTFT P95 by provider (histogram_quantile)
7. RAG retrieval score anomaly (anomaly_score)
8. Top 5 tenants by cost (topk + increase)
9. GPU efficiency: tokens per watt (metric division)

Lab 3 - Configure a production alert

Goal

Configure the RAGIndexDrift alert in VMAAlert. Test it by injecting a score degradation.
Verify the notification in the test Slack channel.
Estimated time: 25 minutes | Level: intermediate

10. Copy and adapt alerts/llm-quality.yml with your test Slack webhook
11. Reload VMAAlert: `docker compose restart vmaalert`
12. Inject a score degradation: `python simulate_llm_metrics.py --score-drift 0.5`
13. Verify in VMAAlert UI (<http://localhost:8880>) that the alert fires after 15 min
14. Confirm receipt in Slack

Lab 4 - Recording rules and performance

Goal

Add the 5 essential recording rules. Compare query response times with and without recording rules via the VictoriaMetrics API.
Estimated time: 20 minutes | Level: advanced

14. References and Resources

Official VictoriaMetrics documentation

Resource	URL
VictoriaMetrics Documentation	docs.victoriametrics.com
MetricsQL Reference	docs.victoriametrics.com/metricsql
vmagent	docs.victoriametrics.com/vmagent
vmalert	docs.victoriametrics.com/vmalert
vmauth	docs.victoriametrics.com/vmauth
vmanomaly	docs.victoriametrics.com/anomaly-detection
GitHub VictoriaMetrics	github.com/VictoriaMetrics/VictoriaMetrics

Integrations and tools

Tool	URL / Reference
OTel Collector Contrib	github.com/open-telemetry/opentelemetry-collector-contrib
DCGM Exporter (GPU)	github.com/NVIDIA/dcgm-exporter
vLLM Metrics	docs.vllm.ai/en/latest/serving/metrics.html
LiteLLM Proxy Metrics	docs.litellm.ai/docs/proxy/prometheus
Grafana VM Plugin	grafana.com/grafana/plugins/victoriametrics-metrics-datasource
prometheus-client Python	github.com/prometheus/client_python

Labs and source code

- github.com/your-org/llm-observability-lab : complete VM + OTel + Grafana labs for LLM
- [llm-observability-lab/labs/victoriametrics-llm](https://github.com/llm-observability-lab/labs/victoriametrics-llm) : ready-to-use Docker Compose stack
- [llm-observability-lab/grafana/dashboard-llm-prod.json](https://github.com/llm-observability-lab/grafana/dashboard-llm-prod.json) : importable Grafana dashboard
- [llm-observability-lab/alerts/](https://github.com/llm-observability-lab/alerts/) : production VMAAlert rules

training courses

Training	Duration	Format
VictoriaMetrics: deployment and administration	2 days	Certified VictoriaMetrics
LLM and RAG observability in production with OTel	2 days	In-house / inter-company
MetricsQL advanced and LLM alerting	1 day	Specialised module
eBPF for advanced observability	3 days	In-house / inter-company

Contact: contact@your-org.example | your-org.example

15. Production case studies

This section presents three real production architectures deployed between 2024 and 2026. Each case combines VictoriaMetrics with a different LLM stack and illustrates real choices around cardinality, retention and alerting.

15.1 Case 1: internal RAG support chatbot (800-employee SMB)

Context: tier-1 support assistant based on Mistral 7B hosted internally, weekly indexing of 12,000 Confluence documents, 600 requests/day.

Metrics stack

- VictoriaMetrics Single (1 VM, 4 vCPU / 8 GB RAM, 250 GB SSD)
- OTEL Collector as sidecar to the LLM API
- Grafana OSS, 1 dashboard, 1 read-only user for management
- VMAAlert with 4 rules, Microsoft Teams webhook output

Observed cardinality

Metric	Cardinality	Notes
llm_request_duration_ms	120 series	model x pipeline x status
rag_retrieval_score_avg	8 series	1 per index_id
llm_tokens_total	60 series	model x type
DCGM_FI_DEV_GPU_UTIL	4 series	2 L4 GPUs
TOTAL ACTIVE	~ 4,200 series	Far below Single mode limits

Lessons learned

- No need for Cluster mode for at least 6 months, Single handles the load.
- Recording rules are essential for hourly cost (otherwise Grafana slows down at month end).
- Most valuable alert: RAGIndexDrift after each Confluence reindexing.

15.2 Case 2: multi-tenant LLM platform (B2B SaaS, 40 customers)

Context: conversational agent platform leased to 40 companies, mix of OpenAI / Anthropic / internal Llama 3 70B model on 8 H100 GPUs.

Metrics stack

- VictoriaMetrics Cluster (3 vmstorage, 2 vminsert, 2 vmselect)
- Dedicated vmagent per zone (EU-West, US-East)
- Grafana Enterprise with per-tenant Teams and RBAC
- VMAAlert: 14 rules including 5 per tenant (budget, latency, quality)

Critical label choices

Key decision

tenant_id only on cost and quality metrics. On latency and error metrics, aggregate by environment (prod / staging) and model. This divides cardinality by 8.

Retention policy	Metrics	Justification
------------------	---------	---------------

3 years	llm_cost_usd_total, llm_tokens_total	Billing, accounting audit
12 months	rag_retrieval_score_avg, hallucination_rate	Long-term quality drift detection
3 months	DCGM_*, vllm_*, raw latency	Short-term capacity planning
7 days	*_debug, *_temp	Incident investigation

15.3 Case 3: batch evaluation pipeline (R&D laboratory)

Context: ML team evaluating 12 prompt variants per week on 5 candidate models, 50,000 prompts per run, exports to Phoenix/Arize for LLM traces.

Specific architecture

- VictoriaMetrics Single in push mode (vmagent receives batches via remote_write)
- No Grafana, dashboards generated in Python (Plotly) after each run
- Run metadata in labels: run_id, prompt_template_id, model_version
- Short retention (90 days), daily export to BigQuery for long-term analysis

Anti-pattern to avoid

NEVER push run_id directly as a label on real-time metrics. Use an external info-style metric llm_run_metadata{run_id, prompt_template_id} = 1 and join with group_left() in MetricsQL.

16. Migrating from Prometheus

VictoriaMetrics is a drop-in replacement for Prometheus in 95% of cases. This section describes the migration in 5 steps, with no service interruption and no data loss.

16.1 Pre-migration inventory

- List all current Prometheus sources (scrapers, remote_write).
- Measure current cardinality: `promtool tsdb stats /var/lib/prometheus/data/`
- List all Grafana dashboards pointing to Prometheus.
- List all Prometheus rules (alerting + recording).
- Identify queries using advanced PromQL functions (rare but possible).

16.2 Dual-write strategy (zero downtime)

For 2 to 4 weeks, write metrics simultaneously to Prometheus and VictoriaMetrics. This lets you compare both backends with no risk.

```
# prometheus.yml: add remote_write to VictoriaMetrics
remote_write:
  - url: http://victoriametrics:8428/api/v1/write
    queue_config:
      capacity: 10000
      max_shards: 30
      max_samples_per_send: 5000
      batch_send_deadline: 5s
```

16.3 Historical data import from Prometheus

To preserve history without starting fresh, use `vmctl`:

```
# Direct import from Prometheus data dir
vmctl prometheus \
  --prom-snapshot=/var/lib/prometheus/snapshots/20260101T000000Z-abc \
  --vm-addr=http://victoriametrics:8428 \
  --vm-concurrency=8 \
  --vm-batch-size=10000

# Verify in vmui:
# http://victoriametrics:8428/vmui/?g0.expr=count(up)
```

16.4 Grafana dashboard migration

Two options based on dashboard fleet complexity:

- Simple option: change the URL of the existing Prometheus datasource. 100% of dashboards keep working (PromQL is a subset of MetricsQL).
- Recommended option: add a new VictoriaMetrics datasource (native plugin), migrate dashboards one by one and remove the Prometheus datasource when dual-write stops.

16.5 Final cutover and measured benefits

Metric	Before (Prometheus)	After (VictoriaMetrics)	Gain
RAM used	14 GB	3 GB	-78 %
Disk (90d retention)	120 GB	16 GB	-86 %

Query latency P99	850 ms	180 ms	-78 %
Supported cardinality	1.2 M (saturates)	10 M+ (stable)	x 8
Monthly infra cost	780 EUR	190 EUR	-75 %

Customer feedback (private bank, Brussels)

Migration of 4 Prometheus instances (federation) to 1 VictoriaMetrics Cluster. Total time: 6 weeks, of which 4 in dual-write. Zero incidents, annual savings of 7,200 EUR on monitoring infrastructure.

17. SLO and SLA for LLM workloads

A Service Level Objective (SLO) translates a business requirement into a measurable metric. For LLMs, classic SLOs (uptime, latency) still apply but are not enough: you must add quality, cost and freshness.

17.1 5-category LLM SLO model

Category	Indicator (SLI)	Typical SLO	Window
Availability	$1 - \text{rate}(\text{llm_errors}[5m]) / \text{rate}(\text{llm_requests}[5m])$	99.5 %	30 days
Interactive latency	$\text{histogram_quantile}(0.95, \text{llm_ttft_ms})$	< 1,200 ms	7 days
Batch latency	$\text{histogram_quantile}(0.99, \text{llm_request_duration_ms})$	< 8,000 ms	30 days
Retrieval quality	$\text{avg}(\text{rag_retrieval_score_avg})$	> 0.78	24h
Cost per request	$\text{increase}(\text{llm_cost_usd_total}[1h]) / \text{increase}(\text{llm_requests_total}[1h])$	< 0.012 EUR	Calendar month

17.2 Error budget calculation

The error budget is the complement to 100% of the SLO. For 99.5% availability over 30 days, the error budget is 0.5% which equals 3h36 of tolerated downtime per month.

Error budget consumed over the SLO window (in %)

```
(
  1
  -
  (
    sum(increase(llm_requests_total[30d]))
    -
    sum(increase(llm_errors_total[30d]))
  )
  /
  sum(increase(llm_requests_total[30d]))
) / 0.005 * 100
```

17.3 Burn rate alerts (Google SRE pattern)

Alert when the error budget is being consumed too fast, with two windows to reduce false positives:

```
- alert: LLMFastBurn
  expr: |
    (
      sum(rate(llm_errors_total[1h])) / sum(rate(llm_requests_total[1h]))
    ) > (14.4 * 0.005)
    and
    (
      sum(rate(llm_errors_total[5m])) / sum(rate(llm_requests_total[5m]))
    ) > (14.4 * 0.005)
  for: 2m
  labels:
    severity: critical
  annotations:
    summary: "Error budget burn rate 14.4x normal (exhausted in 2 days)"
```

```
- alert: LLMSlowBurn
  expr: |
  (
  sum(rate(llm_errors_total[6h])) / sum(rate(llm_requests_total[6h]))
  ) > (6 * 0.005)
  for: 15m
  labels:
  severity: warning
```

18. FAQ and troubleshooting

Q1. VMAAlert does not fire even with an obvious metric.

Check 3 things: (1) is the rule actually loaded? GET `http://vmalert:8880/-/reload` then `http://vmalert:8880/api/v1/rules`. (2) does the query return any series? Test in vmui. (3) is the 'for:' attribute too long? An alert with for: 30m waits 30 minutes.

Q2. Cardinality explodes after adding a new service.

Run GET `http://victoriametrics:8428/api/v1/status/tsdb` to identify top labels. Often a `trace_id` or `request_id` label has leaked. Quick fix: `vmagent --remoteWrite.relabelConfig` to drop the label.

Q3. My recording rules do not update.

VMAlert reloads rules every 60 seconds by default. Force reload: `curl -X POST http://vmalert:8880/-/reload`. Check logs for 'failed to load rule'.

Q4. Difference between vmagent and vmauth?

vmagent = scraping and remote_write client (Prometheus equivalent on the collection side).
vmauth = reverse proxy with authentication and per-tenant routing (useful for multi-tenant and air-gap).

Q5. How do I know when to switch to Cluster mode?

Three signals: RAM used > 70% of host, P99 query > 3 seconds, disk filling up despite compression. Beyond that, plan a Cluster migration over 2 weeks.

Q6. My Prometheus histograms compute P99 incorrectly.

Check the buckets: if the real P99 exceeds the last finite bucket, the result is +Inf or clamped. For LLMs, plan buckets up to 30 seconds: `[.05, .1, .25, .5, 1, 2.5, 5, 10, 30]`.

Q7. I lose points in remote_write from Prometheus.

Tune `queue_config` in `prometheus.yml`: `capacity`, `max_shards`, `max_samples_per_send`. Watch `prometheus_remote_storage_samples_dropped_total`. If non-zero, raise `max_shards`.

Q8. How do I export my data to a 3rd-party platform (DataDog, New Relic)?

VictoriaMetrics exposes `/api/v1/export` in JSON. For continuous exports, use vmagent with two remote_write: one to VM, one to the 3rd party. Watch out for cost: each sample is double-billed.

Q9. My Unicode labels (Chinese, accents) break queries.

VictoriaMetrics supports UTF-8 but Grafana may render poorly. Wrap values in back-ticks and prefer ASCII labels for business values (e.g., `tenant_id` as `slug`).

Q10. My dashboard loads in 10 seconds after adding a variable.

A Grafana variable = a sub-query on every refresh. Solutions: (1) cache the variable in Grafana (refresh on dashboard load), (2) pre-aggregate via recording rule, (3) limit options with `sort + limit`.

Q11. A GPU disappears from DCGM exporter metrics.

DCGM exporter uses NVML which can lose the GPU list if the driver crashes. Check `nvidia-smi` on the node, restart `dcm-exporter` and `nvidia-persistenced`. For prevention: alert on `absent(DCGM_FI_DEV_GPU_UTIL{node='X'})`.

Q12. How do I version my VMAalert alerts?

Store YAML files in Git, use ArgoCD or Flux for deployment, vmaalert reads files from a volume. Test each PR with promtool check rules.

Q13. Difference between rate() and increase() for costs?

rate(llm_cost_usd_total[5m]) returns USD per second. increase(llm_cost_usd_total[1h]) returns the total delta over 1h in USD. For displayed hourly cost, prefer increase. For projection, prefer rate.

Q14. My alerts send duplicates to Slack.

Check Alertmanager configuration: group_by, group_wait, group_interval. For LLMs, group_by: ['alertname', 'model'] groups alerts per model, repeat_interval: 4h prevents spam.

Q15. Point-in-time backup: which strategy?

vmbackup to S3-compatible (MinIO in air-gap). Daily incremental snapshots, 30-day retention. Mandatory monthly restore test. Typical storage cost: 5% of live data.

Appendix A. Glossary

Anomaly score: MetricsQL function that measures how far a series deviates from a historical baseline. Value > 1 = anomaly.

Burn rate: Speed at which the error budget is consumed. Burn rate = 1 = linear consumption over the SLO window.

Cardinality: Number of unique time series (metric + label combinations).

Chunk: Document fragment indexed in a vector database for RAG.

Context pressure: Ratio of used tokens to the model's context window. > 0.85 = truncation risk.

Counter: Metric type that only increases (except on reset). Ex: total requests.

DCGM: NVIDIA Data Center GPU Manager. Source of real-time GPU metrics.

Embedding: Vector representation of text computed by an embedding model.

Error budget: Tolerated error margin allowed by the SLO over a given window.

Fan-out: Cardinality multiplier resulting from label combinations.

Gauge: Metric type that can go up and down. Ex: GPU utilization.

Hallucination rate: Estimated rate of non-factual responses, measured by evaluators or heuristics.

Histogram: Metric type that distributes values into buckets to compute quantiles.

Index drift: Drift in retrieval score after reindexing, sign of an embedding or corpus change.

KV cache: Cache of attention keys/values in a Transformer model during inference.

Latency P99: 99th percentile of latency: 99% of requests are faster.

LiteLLM: Unified proxy that exposes OpenAI, Anthropic, Mistral etc. APIs behind a common interface.

MetricsQL: VictoriaMetrics query language, superset of PromQL.

OpenTelemetry (OTel): Instrumentation standard for traces, metrics and logs.

OTLP: OpenTelemetry Protocol: binary (gRPC) or JSON (HTTP) format for exporting telemetry.

Prompt template: Prompt skeleton with variables, versioned like code.

RAG: Retrieval Augmented Generation: enriching a prompt with retrieved documents.

Recording rule: Rule that pre-computes a derived metric and stores it as a new series.

Remote write: Protocol for pushing metrics from a Prometheus/vmagent to a remote TSDB.

Retrieval score: Similarity score (0 to 1) between a question and the chunks returned by RAG.

Single mode (VM): VictoriaMetrics deployment as a single binary, up to 1M active series.

Cluster mode (VM): VictoriaMetrics deployment in 3 components (vminsert, vmstorage, vmselect) for horizontal scale.

SLI: Service Level Indicator: the metric being measured.

SLO: Service Level Objective: internal target on the SLI.

SLA: Service Level Agreement: contractual commitment with an external customer.

Tenant: Logical customer of a multi-tenant platform, isolated by tenant_id.

Token: Text-splitting unit for an LLM (sub-word), billed by providers.

TSDB: Time Series Database. VictoriaMetrics, Prometheus, InfluxDB are TSDBs.

TTFT: Time To First Token: latency before the first token of a streamed response.

vLLM: High-performance open-source inference engine for LLMs.

VMAgent: VictoriaMetrics component that scrapes and does remote_write.

VMAlert: VictoriaMetrics component that evaluates alerting and recording rules.

VMAuth: VictoriaMetrics reverse proxy with authentication and per-tenant routing.

VMSelect: VictoriaMetrics Cluster component that serves queries.

VMStorage: VictoriaMetrics Cluster component that stores data.

VMUI: VictoriaMetrics built-in web UI (vmui).

VRAM: Video memory of a GPU (HBM, GDDR). Limits the loadable model size.

APPENDIX B

Appendix B. MetricsQL cheat sheet for LLMs

Cost and tokens

```
# Hourly cost per model
sum by(model) (increase(llm_cost_usd_total[1h]))

# Projected daily cost
sum(rate(llm_cost_usd_total[30m])) * 86400

# Top 10 tenants over 24h
topk(10, sum by(tenant_id) (increase(llm_cost_usd_total[24h])))

# Average cost per request (recording rule recommended)
sum(rate(llm_cost_usd_total[5m])) / sum(rate(llm_requests_total[5m]))
```

Latency and performance

```
# P99 end-to-end latency per model
histogram_quantile(0.99, sum by(le, model) (rate(llm_request_duration_ms_bucket[5m])))

# TTFT P95 (streaming)
histogram_quantile(0.95, sum by(le, model) (rate(llm_ttft_ms_bucket[5m])))

# Global error rate
sum(rate(llm_errors_total[5m])) / sum(rate(llm_requests_total[5m]))

# Throughput tokens/s per model
sum by(model) (rate(llm_tokens_total{type='output'}[1m]))
```

RAG quality

```
# Retrieval score anomaly (7-day baseline)
anomaly_score(avg by(index_id) (rag_retrieval_score_avg), "7d")

# P95 context pressure per pipeline
histogram_quantile(0.95, sum by(le, pipeline_id)
(rate(rag_context_pressure_ratio_bucket[5m])))

# Dropped chunks rate (information loss)
sum by(pipeline_id) (rate(rag_chunks_dropped_total[5m]))
/ sum by(pipeline_id) (rate(rag_chunks_retrieved_total[5m]))
```

GPU and infrastructure

```
# Average GPU utilization per node
avg by(node, gpu) (DCGM_FI_DEV_GPU_UTIL)

# VRAM saturation (alert before OOM)
DCGM_FI_DEV_FB_USED / DCGM_FI_DEV_FB_TOTAL

# Tokens per watt (efficiency)
sum by(model) (rate(llm_tokens_total{type='output'}[5m]))
/ sum by(model) (avg(DCGM_FI_DEV_POWER_USAGE))

# vLLM queue
```

```
vllm:num_requests_waiting
```

Advanced patterns

```
# Robust median (ignores spikes)
median_over_time(llm_tokens_per_request_avg[30m])

# OHLC on RAG scores (candlestick visualization)
rollup_candlestick(rag_retrieval_score_avg[1h])

# Plateau detection (rate of change < 1%)
abs(deriv(llm_requests_total[10m])) < 0.01

# Week-over-week comparison
sum(rate(llm_cost_usd_total[1h]))
/ sum(rate(llm_cost_usd_total[1h] offset 7d))
```

Appendix C. Production go-live checklist

Before go-live

- ☐ VictoriaMetrics is deployed with --storageDataPath on a persistent SSD disk
- ☐ Retention flags are set (--retentionPeriod and --retentionFilter)
- ☐ vmbackup is configured to S3-compatible storage (restore test passed)
- ☐ vmagent or OTel Collector ingests the 4 metric categories (quality, latency, cost, GPU)
- ☐ The 6 production VMAAlert rules are loaded and tested (test alert received in Slack/Teams)
- ☐ At least 2 recording rules are active for costs (Grafana performance gain)
- ☐ The production Grafana dashboard is imported, template variables work
- ☐ vmauth is configured with basic auth or mTLS (never unauthenticated access)
- ☐ Total cardinality is measured and documented (curl /api/v1/status/tsdb)
- ☐ Risky labels (trace_id, session_id) are dropped via relabelConfig
- ☐ The 'VictoriaMetricsDown' (self-monitoring) alert is active
- ☐ An incident runbook exists (who to call, how to restore, how to scale)
- ☐ Read-only Grafana users are created for non-tech roles (management, finance)

After go-live (D+7, D+30)

- ☐ Verify cardinality has stabilized (no linear growth)
- ☐ Verify disk consumption and project to 90 days
- ☐ Receive at least one real alert and confirm routing works
- ☐ Measure P99 latency of Grafana queries (< 1s expected)
- ☐ Run a vmbackup restore test on a test instance
- ☐ Review alert thresholds with teams (false positives?)
- ☐ Document custom MetricsQL queries used by teams